

ENSAM casablanca

Department of IA & Computer Science

DRL for Financial Trading

AFANDI IMANE

KANAS OUMAIMA

Supervisor: Badr Hirchoua

A report submitted in partial fulfilment of the requirements of
the Hassan II University
Artificial Intelligence and Computer Science

November 11, 2026

Declaration

I, AFANDI IMANE and KANAS OUMAIMA, of the Department of Computer Science, University of Reading, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my report being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of UoR and public with interest in teaching, learning and research.

AFANDI IMANE

KANAS OUMAIMA
November 11, 2026

Abstract

Financial trading is a difficult task because market data is noisy and hard to interpret. In this project, we built an automated trading system that combines Deep Reinforcement Learning (DRL) with a Variational Autoencoder (VAE). The idea was to use the VAE to compress raw market data into a cleaner representation before giving it to the trading agent, instead of using the raw feature directly. We collected daily data for the SPY ETF, the VIX index, and the Federal Funds Rate, covering the period from January 2019 into January 2024. After computing 24 features, we trained a VAE with LSTM layers to compress 20-day windows into a 32-dimensional vector. This vector was then used as the input for a SAC agent trained in a custom trading environment. The results showed that our system performed better than both a Buy & Hold strategy and a DRL agent trained without the VAE. This suggest that improving the state representation actually help the agent make better trading decision.

Keywords: Deep Reinforcement Learning, Variational Autoencoder, Financial Trading, State Representation, SAC, SPY ETF, LSTM, Backtesting.

Acknowledgements

We want to say thank you to our professor Badr Hirchoua for teaching us this year. He also gave us a chance to work on this project. The things we learned in his class were really important, for finishing this work.

We also want to thank each other. We worked together on this project. We both tried really hard. our professor taught us a lot of things that helped us with this project. We are glad we got to work with each other on this project.

Contents

List of Figures vi

List of Tables vii

1 Introduction 2
- 1.1 Project Presentation . . . 2
  - 1.1.1 Background . . . 2
  - 1.1.2 Problem Statement . . . 2
  - 1.1.3 Aims and Objectives . . . 2
  - 1.1.4 Solution Approach . . . 3
- 1.2 Summary . . . 3

2 Literature Review 4
- 2.1 DRL methods for algorithmic trading . . . 4
- 2.2 DRL Algorithms: PPO, SAC, and TD3 . . . 4
- 2.3 Variational Autoencoders for State Representation . . . 6
  - 2.3.1 The VAE Framework . . . 6
  - 2.3.2 VAEs for Financial Representation Learning . . . 6
- 2.4 Multi-Source Financial Feature Engineering . . . 7
- 2.5 Critique of the Literature and Identified Gaps . . . 7
- 2.6 Summary . . . 8

3 Methodology 9
- 3.1 Report Structure . . . 9
- 3.2 Ethical Considerations . . . 9
- 3.3 Phase 1 – Data Collection and Feature Engineering . . . 10
  - 3.3.1 Data Sources . . . 10
  - 3.3.2 Technical Indicators . . . 10
  - 3.3.3 Sliding Window and Normalization . . . 10
- 3.4 Phase 2 – VAE Architecture and Training . . . 11
  - 3.4.1 Why a VAE? . . . 11
  - 3.4.2 Architecture . . . 11
  - 3.4.3 Training . . . 12
- 3.5 Phase 3 – Custom Trading Environment . . . 13
  - 3.5.1 Overview . . . 13
  - 3.5.2 State Space . . . 13
  - 3.5.3 Action Space . . . 13
  - 3.5.4 Reward Function . . . 13
  - 3.5.5 Initialization . . . 13

3.6 Phase 4 – DRL Agent Training . . . 13
3.6.1 Algorithm Comparison . . . 13
3.6.2 SAC Results and Hyperparameters . . . 14
3.6.3 Ablation Study . . . 14
3.7 Phase 5 – Backtesting and Evaluation . . . 15
3.8 Summary . . . 15

4 Results **17**
4.1 VAE Training Results . . . 17
4.2 Latent Space Analysis . . . 18
4.3 DRL Algorithm Comparison . . . 18
4.4 Ablation Study Results . . . 19
4.5 Out-of-Sample Backtesting Results . . . 20
4.6 summary . . . 21

5 Discussion and Analysis **22**
5.1 Interpretation of Results . . . 22
5.1.1 VAE as an Effective State Encoder . . . 22
5.1.2 Algorithm Comparison: PPO, SAC, and TD3 . . . 22
5.1.3 Multi-Source Data Integration . . . 23
5.1.4 Backtesting Performance: DRL + VAE vs. Baselines . . . 23
5.2 Analysis of the Ablation Study . . . 23
5.3 Significance of the Findings . . . 24
5.4 Limitations . . . 24
5.5 Summary . . . 25

6 Conclusions and Future Work **26**
6.1 Conclusions . . . 26
6.2 Future Work . . . 26
6.3 Summary . . . 26

7 Reflection **27**

References **28**

List of Figures

4.1 Comparison of algorithm performance across evaluated configurations 18
4.2 performance results of all three strategies. 19
4.3 [Courbe portfolio.]. 20

List of Tables

3.1 Report structure used in this project 9
3.2 Summary of all features used in the model 10
3.3 SAC hyperparameters 14
3.4 Performance metrics used for evaluation 16

4.1 VAE training and validation losses across selected epochs. 17
4.2 Best validation return per DRL algorithm 18
4.3 presents the complete performance results of all three strategies on the test set. 19
4.4 Full backtesting results on the out-of-sample test set (177 days).) 20
4.5 Summary of results mapped to project objectives 21

List of Abbreviations

DRL	Deep Reinforcement Learning
VAE	Variational Autoencoder
SAC	Soft Actor-Critic
PPO	Proximal Policy Optimization
TD3	Twin Delayed Deep Deterministic Policy Gradient
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
OHLCV	Open, High, Low, Close, Volume
RSI	Relative Strength Index
MACD	Moving Average Convergence Divergence
ATR	Average True Range
SMA	Simple Moving Average
EMA	Exponential Moving Average
KL	Kullback-Leibler Divergence
t-SNE	t-Distributed Stochastic Neighbor Embedding
ETF	Exchange-Traded Fund
SPY	SPDR S&P 500 ETF Trust

General Introduction

General Trading in markets is really tough. The prices of things are always changing. It is hard to know what will happen next. In the few years Deep Reinforcement Learning has been used to make trading decisions and it has worked pretty well. But there is a problem. The information from financial markets is messy and there is too much of it which makes it hard for the computer to understand.

In this project we tried to fix this problem by using a Variational Autoencoder to make the market data cleaner and easier to use before we give it to the trading agent. We tested our system using the SPY ETF. We used data from January 2019 to January 2024 including prices the VIX index and the Federal Funds Rate.

The results showed that our system, which uses Deep Reinforcement Learning and a Variational Autoencoder worked better than buying and holding onto things or using Deep Reinforcement Learning without a Variational Autoencoder. This shows that making the data cleaner really helps the agent make decisions.

Organization of the report

This report is divided into seven parts. Chapter 1 is the introduction. It says what we want to do. Chapter 2 looks at what other people have done. Chapter 3 explains how we did things. Chapter 4 shows what we found out. Chapter 5 talks about what it all means. Chapter 6 says what we think and what we can do next. Chapter 7 is what I think about the thing.

Chapter 1

Introduction

This chapter provides the first picture of the project. It introduces the host organization, describes the problem being investigated, outlines the aims and objectives, and summarises the solution approach and project management plan.

1.1 Project Presentation

1.1.1 Background

Financial markets generate massive amounts of trading data every day, which greatly increases the difficulty of trading decision-making. Traditional trading methods that follow fixed rules cannot adapt to the ever-changing market environment. Deep Reinforcement Learning, an artificial intelligence (AI) method capable of autonomously learning strategies from market data, holds strong potential, but it is constrained by the inherent characteristic of financial data: high noise, high complexity, and an abundance of redundant information. These factors cause the difficulty of its learning process to rise sharply.

We use Variational Autoencoders (VAEs) to make the data simpler. VAEs keep only the important information. We combine VAEs with Deep Reinforcement Learning (DRL). This help the trading agent see the market better. It also helps the agent make better trading decisions.

1.1.2 Problem Statement

Financial market data have a lot of noise. They are hard to understand and have many parts. This makes it hard for Deep Reinforcement Learning (DRL) agents to find a good trading way. Using raw market data can give bad information about the market and cause unstable learning. To fix this, we study how to use Variational Autoencoders (VAEs) with DRL. VAEs change complex financial data into simple forms. DRL uses these simple forms to learn the best trading choice. We want to see if this simple form helps the DRL trading agent work better and be more steady.

1.1.3 Aims and Objectives

Aims: We want to create a trading system. It will use Deep Reinforcement Learning or DRL for short. A Variational Autoencoder or VAE will help us understand the data and make trades.

Objectives:

1: We need to get data from many places. This data includes SPY OHLCV market data, the VIX index and interest rates from the Federal Reserve.

2: We will design a special kind of VAE called a temporal LSTM-VAE. It will learn from sequences of data to find hidden pattern.

3: We will make a custom trading environment using OpenAI Gym. This environment will be realistic with things like costs for buying and selling, slippage and limits on how much we can buy or sell.

4: We will test three DRL algorithms: PPO, SAC and TD3. We will use the data from the VAE as input. Find the best one.

5: We will do a study to compare how well DRL with VAE does compared to DRL without VAE and a simple Buy-and-Hold method. We will use data that the algorithms have not seen before.

1.1.4 Solution Approach

The solution was to combine a Variational Autoencoder (VAE) with a Deep Reinforcement Learning (DRL) framework for financial trading. So first we collect and preprocess multi-source financial data, including SPY, VIX, and interest rates, then train a time-series LSTM-VAE to learn compact representations of market states. These latent features are used as input to a DRL trading agent operating in a custom trading environment. Several DRL algorithms are trained and evaluated to identify the most effective trading strategy. Finally, the proposed approach is assessed through backtesting, comparison with baseline methods.

1.2 Summary

In summary, this chapter has introduced the host organization and established the context in which this project is situated. It described the problem statement, the aims and objectives, and the solution approach adopted. The project management plan was also outlined. Chapter 2 presents a review of the existing literature relevant to this project.

Chapter 2

Literature Review

This chapter provides an overview of the available literature on the topics pertaining to the main aspects of this research project: Deep Reinforcement Learning (DRL) techniques used for trading tasks, state representation learning through VAEs, and multi-source financial data integration. The literature overview will be presented thematically in the following manner: Section 2.1 focuse on DRL methods for algorithmic trading; Section 2.2 cover DRL Algorithms: PPO, SAC, and TD3; Section 2.3 Variational Autoencoders for State Representation; Section 2.4 Multi-Source Financial Feature Engineering.

2.1 DRL methods for algorithmic trading

Reinforcement Learning (RL) is a type of machine learning where an agent learn by interacting with an environment, observe its current state, take actions, and get rewards that indicates the quality of the agent's decision. The objective of the agent is to maximise the aded rewards over time by learning an optimal policy. contrary to supervised learning, where labels are required, RL learns based on trial and error, making it the most fitting technique for sequential decision-making problems such as financial trading. In this context, the trading agent have to decide to buy, sell, or hold assets while considering both immediate profits and long-term performance .

Initial approaches for applying reinforcement learning into trading involved tabular RL algorithms suh as Q-Learning and SARSA.Even thought these approaches proved that it was possible to learn how to trade using the interactions with the market, they lacked scalability because of reliance on action spaces. With the massive amount of high dimensional and continuous data that is produced in financial markets, the initial reinforcement learning algorithms face difficulties in working with the dimensionality (Sutton and Barto, 2018).

the development of Deep Reinforcement Learning (DRL) gave a solution to a lot of the limitaions of machine learning by combining reinforcement learning with deep neural networks. One category of DRL approaches that has shown promise in terms of learning effective trading strategies based on market data is value-based models like the Deep Q-Network (DQN). Nevertheless, it appears that approaches based on DQN algorithms may only be applicable in discrete action spaces (Mnih et al., 2015).

2.2 DRL Algorithms: PPO, SAC, and TD3

Mltiple studies showed a lot of promising results. Algorithms like Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3)

employ separate actor and critic networks to improve learning stability and policy optimisation.

- **Proximal Policy Optimization (PPO)** : The Proximal Policy Optimization algorithm (PPO), proposed by Schulman et al. (2017), is an on-policy actor-critic approach that boosts training efficiency by bounding the policy update using the clipping mechanism. PPO has been established as one of the most popular DRL algorithms because of its simple structure, stability, and generalizability to both continuous and discrete action spaces. For applications in financial trading, the reliability of the convergent solution obtained through PPO is another advantage of the algorithm. However, since the algorithm is an on-policy approach, it discards the gathered experience after each update step. This makes the approach less efficient than off-policy approaches.

- **Soft Actor-Critic (SAC)** : Soft Actor-Critic (SAC), proposed by Haarnoja et al. (2018), is a method for off-policy learning based on the maximum entropy RL paradigm. In addition to maximising the expected reward, SAC maximises entropy over its policy in order to ensure exploration and avoid early convergence to sub-optimal deterministic solutions. SAC implements twin Q networks to address the problem of overestimation and adjusts the entropy coefficient autonomously through a target entropy condition. SAC's entropy-based approach fits well into the problem of trading in financial markets, where the optimal solution can be stochastic. In practice,Rezaei and Nezamabadi-Pour (2025)report that SAC obtains the highest Sharpe Ratio across five DRL methods used for portfolio optimisation of a DJIA benchmark.

- **Twin Delayed Deep Deterministic Policy Gradient (TD3)** : TD3, developed by Fujimoto et al. (2018), enhances DDPG through three important developments: (1) twin critics for solving the problem of Q-value overestimation; (2) delayed policy updates compared to those of critics; and (3) smoothed target policy through the addition of noise. The stability mechanisms resolve the problem of instability and overestimation encountered in the original version of DDPG. The continuous-action trade algorithm TD3 offers a deterministic policy, which could be an advantage when exact position sizing is essential. Nevertheless, its deterministic nature ight result in inadequate exploration in the presence of noisy marketsm.

It is important that the three methods, PPO, SAC, and TD3, are implemented with the same market settings and representation of states to have a valuable comparison. Indeed, some experiments are considered as best practices in deep reinforcement learning trading literature (Felizardo et al., 2022) because they make it possible to identify the performance of the algorithm separate from the environmental setup and feature development.

A study realised by Rezaei and Nezamabadi-Pour (2025), that was published in Artificial Intelligence Review, rate five actor-critic DRL algorithms which are A2C, DDPG, PPO, SAC, and TD3 throught multiple financial benchmarks. The study showed that off policy methods espacially SAC outperform on-policy approaches like PPO in continuous-action financial tasks, owing to SAC's entropy regularisation which gives premature convergence to sub optimal deterministic policies. This study supports the algorithm selection that's gonna be presented in our project, where SAC achieved the highest validation return compared to the three algorithms.

Eventhought these advances, Pricope (2021) points out important challenges: a lot of published DRL trading studies depend on unrealistic assumptions, like zero transaction costs, no slippage, or in-sample evaluation.

2.3 Variational Autoencoders for State Representation

A fundamental challenge in applying DRL to financial markets was the quality of the state representation communicated to the agent. Unit price data and technical indicators are multi featured ,high dimensional, noisy, and strongly correlated, which can impede learning. Representation learning attemps to extract compact, informative, and disentangled features from raw observations.

.

2.3.1 The VAE Framework

The Variational Autoencoder (VAE), proposed by Kingma and Welling (2013), is a generative latent variable model that encodes input data into a continuous probabilistic latent space by maximising evidence lower bound (ELBO), which combines a reconstruction loss and a KL-divergence regularisation term. Unlike standard autoencoders, the VAE's latent space is smooth and structured, allowing meaningful interpolation and generalisation. The reparameterisation trick allows allows training the VAE using gradients despite the stochastic nature of sampling.

The relevance of temporal architectures for financial encoding is further confirmed by a large-scale benchmark from the University of Oxford(Saly-Kaufmann et al., 2026), which evaluates modern deep learning architectures on a daily futures dataset covering betweeen 2010 and2025. The study find that models that were assigned to learn rich and various temporal representations like hybrid LSTM-based architectures outperform both linear benchmarks and generic deep learning models in terms of Sharpe ratio and downside-risk metrics. This gives a independent convincing justification to use a two layer LSTM encoder within the VAE architecture adopted in this project.

There are two implications of the latent space being probabilistic for this project. First, it results in a manifold being smooth and continuous, making the learned representation resistant to noise present in financial data. Second, since the encoder cannot rely on the KL penalty, it has to extract useful features from input data, rather than simply memorise the input values.

2.3.2 VAEs for Financial Representation Learning

For sequential financial data, feedforward VAE is not sufficiant since it does not capture any temporal dynamics. Recurrent VAE, that uses the encoder and decoder using LSTM or GRU networks, have showed more suitable for modeling multivariate time-series ((Fabius and van Amersfoort, 2014); (Chung et al., 2015). The VRNN, introduced by (Chung et al., 2015), models temporality explicitly using latent variables at each time step. For this project, an encoder consisting of a two-layer LSTM network is used, where the inputs consist of 20 consecutive time steps for 26 features, outputting latent vectors of size 32, which is uesd as the input for DRL states. The motivation of this selection lcomes because of the balancing temporal compression and computational efficiency in a gymnasium setting.

- The use of VAEs as state encoders for RL was explored in robotics ((Higgins et al., 2017); (Ha and Schmidhuber, 2018), where a lot of models framework showed that a VAE can compress visual observations into compact representations that enable efficient policy learning. In finance, Lussange et al. (2021) and Zhang et al. (2020)

explored autoencoder-based feature extraction for trading agents, but without the probabilistic regularisation of a VAE. This project addresses this gap by using a VAE with KL weighting ($\beta = 0.5$) to produce a regularised, continuous latent space from multi-source financial inputs.

2.4 Multi-Source Financial Feature Engineering

The quality of the state representation is determined by how rich the input features are. Financial data can be categorized into three broad types of financial features: price-based features, volatility indicators, and macroeconomic indicators. Price-based technical indicators represent the largest group of indicators utilized in algorithmic trading. Technical indicators such as moving average (SMA, EMA), relative strength index (RSI), momentum oscillators, Bollinger bands, and average true range (ATR) have been found to provide predictive content in different regimes (Brock et al., 1992) ; (Murphy, 1999). In this project, 18 technical indicators are calculated using SPY OHLCV data, where 6 features related to returns for 1-day, 5-day, and 20-day periods are used to capture momentum at different frequencies. Volatility signals,especially the CBOE Volatility Index (VIX), are commonly employed as proxies for risk aversion and uncertainty across markets. Macroeconomic variables, in particular the Federal Funds Rate from FRED (Federal Reserve Bank of St. Louis), capture the monetary policy regime that influences equity valuations through the discount rate channel. Changes in interest rates affect equity valuations in known ways (Bernanke and Kuttner, 2005). Adding the Fed Rate, its first differences, and a 3-period moving average will enable the agent to better interpret the context of changes in equity price as being influenced by monetary policy decisions. Taken together, the combination of these three sets of data into a 26-variable vector to feed to the VAE is a purposeful decision based on their complementarities: price captures market microstructure, VIX captures macro fear, while the Fed Rate captures the monetary regime.

2.5 Critique of the Literature and Identified Gaps

The literature review shows several key gaps that motivate the choices in this project. First, a lot of DRL trading systems are based on raw or hand-engineered features directly as the agent observation, without a compression step. This results in high-dimensional, noisy states that slow convergence and reduce generalisation. Second, existing work on VAE-based state encoding for trading does not exploit multi-source heterogeneous inputs, limiting the informational richness of the latent representation. Third, comparative studies of PPO, SAC, and TD3 on a unified, cost-aware trading environment with VAE-encoded states have not been conducted, leaving the relative strengths of these algorithms in this setting unexplored. This project addresses all three gaps by: (1) using an LSTM-VAE to produce a compact, regularised latent state from 26 heterogeneous features; (2) fusing OHLCV, VIX, and macroeconomic data into a single representation; and (3) systematically comparing PPO, SAC, and TD3 under identical conditions in a custom Gymnasium environment with a realistic reward function.

To add the ' using an LSTM-VAE to produce a compact, regularised latent state from 26 heterogeneous features' limitation is still an active subject of research: Li et al. (2026) proposed CE-PPO, a framework published in January 2026 that clearly acknowledge the over reliance of DRL trading models on raw market data and addresse it by adding cluster embeddings to the state from feature channel. While CE-PPO improves by raw-feature baselines, it doesn't employ a probabilistic latent space, or integrate multi-source heterogeneous data, nor apply

temporal compression via a recurrent encoder.

2.6 Summary

This chapter reviewed the existing literature on DRL for financial trading, temporal feature extraction with LSTMs, VAE-based state representation, and the PPO, SAC, and TD3 algorithms. Three key gaps were identified: the lack of multi-source data integration in state representation, the use of non-temporal VAE encoder, and the absence of systematic multi-algorithm ablation studies. This project addresses all three gaps through its architecture: a multi-source LSTM-VAE that compresses 27 feature from a 20-day window into a 32-dimensional latent space, subsequently used as the state for PPO, SAC, and TD3 agent in a realistic Gymnasium trading environment with transaction costs and slippage. Chapter 3 describe the methodology used to implement and evaluate this system.

Chapter 3

Methodology

This chapter is about the projects methodology. It talks about how the system was developed from getting the data to checking the trading agent. The project is set up in a way that uses a set of algorithms and tools to solve a problem, which is automatic financial trading using Deep Reinforcement Learning and a Variational Autoencoder.

3.1 Report Structure

This project follows the Application Type report structure (see Table 3.1 of the template), since the main goal is to apply algorithms (VAE and SAC) to a real problem (trading on the SPY). The methodology chapter is therefore organized around the five main phases of the project.

Table 3.1: Report structure used in this project

Phase	Content
Phase 1	Data collection and feature engineering
Phase 2	VAE architecture and training
Phase 3	Custom trading environment (OpenAI Gym)
Phase 4	DRL agent training (PPO, SAC, TD3) and ablation study
Phase 5	Backtesting and performance evaluation

3.2 Ethical Considerations

This project is something we did for school. We used money and did not actually buy or sell anything.

We got all the information we used like the prices of SPY, VIX and the Fed Rate from places where anyone can get it for free. We did not collect any information about people.

We are telling the truth about what we found out in this report. We did not leave out the parts or make up numbers to make our system seem better than it really is.

Finally we made this system just to learn. You should not use it to make real decisions, about money unless you test it carefully and think about the risks of the Ethical Considerations of the system the Ethical Considerations of the project and the Ethical Considerations of our research.

3.3 Phase 1 – Data Collection and Feature Engineering

3.3.1 Data Sources

We used three data sources covering January 2019 to January 2024.
- The first is the SPY ETF,which tracks the S&P 500, downloaded via yfinance.
- The second is the VIX index (the "fear index"), which measures market uncertainty.
- The third is the Federal Funds Rate from the FRED database since it is monthly, we converted it to daily by repeating the last known value.

We merged the three sources into one table and removed any remaining missing values. The final dataset contains around 1,250 trading days

3.3.2 Technical Indicators

Raw price data alone is usually not enough for a trading system, so we added a set of technical indicators that are commonly used to detect trends and patterns in the market.Table 3.2 shows all the features we used.

Table 3.2: Summary of all features used in the model

Category	Feature	Description
Returns	return_1d, return_5d, return_20d	Price changes over 1, 5, and 20 days
Moving Averages	sma_10, sma_50, ema_12, ema_26	Simple and exponential moving averages
MACD	macd, macd_signal, macd_hist	Trend-following momentum indicator
RSI	rsi_14	Relative Strength Index
Bollinger Bands	bb_upper, bb_lower, bb_width, bb_pct	Volatility bands around price
ATR	atr_14	Average True Range — daily volatility
Volume	vol_sma20, vol_ratio	Volume relative to its 20-day average
VIX	vix and derived features	Market fear index and derived features
Fed Rate	fed_rate and derived features	Interest rate and its recent trend

The total number of features is 24. These features are from places like price and volatility and macro. This means the features show us things about the market. The fact that we have many different features is one of the reasons we used the VAE. The VAE helps us to make sense of all these features by turning them into something simpler which is the latent representation of the market. We use the VAE to do this because it is good, at taking a lot of information and making it easier to understand.

3.3.3 Sliding Window and Normalization

We used sliding windows of 20 trading days.Each window has the 20 days of all 24 features.This gives us an array with the shape of (samples, 20 24).The LSTM, inside the VAE can learn patterns in the data over time.We then split the data into three parts:

- 70% for training
- 15% for validation
- 15% for testing

We applied Z-score normalization to the data.The mean and standard deviation were calculated from the training set.These values were then used on the validation and test sets.This is a practice to prevent data leakage

Algorithm 1 Sliding Window Creation and Normalization

Input: df (N days, 24 features), $WINDOW = 20$
Output: X_{train}, X_{val}, X_{test} (normalized)

// Step 1: Create sliding windows
$X \leftarrow \emptyset$
for $i \leftarrow WINDOW$ to N **do**
    $window \leftarrow df[i - WINDOW : i]$
    Add $window$ to X
end for

// Step 2: Split
$X_{train} \leftarrow X[0 : 70\%]$
$X_{val} \leftarrow X[70\% : 85\%]$
$X_{test} \leftarrow X[85\% : 100\%]$

// Step 3: Normalize (fit on train only)
Fit scaler on X_{train}
$X_{train} \leftarrow scaler.transform(X_{train})$
$X_{val} \leftarrow scaler.transform(X_{val})$
$X_{test} \leftarrow scaler.transform(X_{test})$

return X_{train}, X_{val}, X_{test}

3.4 Phase 2 – VAE Architecture and Training

3.4.1 Why a VAE?

The input data we have is really big and messy. It has 24 features that are collected over 20 days. This is a lot of information. It is not easy to work with. If we give this data to the trading agent directly it will have a time learning from it. The learning process will be slow. It will not be stable. That is why we used a Variational Autoencoder or VAE for short. The VAE helps us to compress this input into a smaller vector that is 32 units long. The idea behind this is that the compressed version will keep the important information and get rid of the noise.

3.4.2 Architecture

The VAE has two parts: the encoder and the decoder.

The encoder looks at the data from the 20-day window. Uses a special kind of neural network called a 2-layer LSTM to make sense of it. After that it creates two vectors that're 32 units long: the mean and the log-variance. We use these two vectors to create a latent vector z. We do this by using the following formula:

$$z = \mu + \epsilon \times \exp(0.5 \times \log_var) \tag{3.1}$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

The decoder then takes z and tries to reconstruct the original input sequence.

3.4.3 Training

The VAE is trained by minimizing two things at the same time: how well it reconstructs the input, and how close the latent distribution is to a standard Gaussian. This is the ELBO loss:

$$L = MSE(x_{reconstructed}, x) + 0.5 \times KL(q(z|x) \,||\, p(z)) \tag{3.2}$$

We trained for 50 epochs using the Adam optimizer with a learning rate of 0.001. After training, we used the encoder to compress all the data into latent vectors Z of shape (samples, 32), which were then passed to the trading agent.

Algorithm 2 VAE Training

Input: X_{train}, X_{val}, $epochs = 50$, $batch_size = 64$
Output: Best VAE model

Initialize VAE (Encoder LSTM + Decoder LSTM)
Initialize Adam optimizer, $lr = 0.001$
Initialize ReduceLROnPlateau scheduler
for $epoch \leftarrow 1$ to 50 **do**

    // **Training**
    for each batch x in $train_loader$ **do**
        $x_{reconstructed}, \mu, logvar \leftarrow VAE(x)$
        $recon_loss \leftarrow MSE(x_{reconstructed}, x)$
        $kl_loss \leftarrow -0.5 \times mean(1 + logvar - \mu^2 - exp(logvar))$
        $loss \leftarrow recon_loss + 0.5 \times kl_loss$
        Update weights using Adam
    end for

    // **Validation**
    for each batch x in val_loader **do**
        Compute val_loss (no gradient update)
    end for
    Update learning rate using scheduler
    if $val_loss < best_val_loss$ **then**
        $best_val_loss \leftarrow val_loss$
        Save model checkpoint
    end if
end for
return Best model

3.5 Phase 3 – Custom Trading Environment

3.5.1 Overview

We made a trading environment using the OpenAI Gym framework. This framework is a tool for reinforcement learning. It shows how the agent interacts with the market. The agent sees what is happening decides what to do and gets a reward.

3.5.2 State Space

Each time the agent takes a step it gets some information. This information has 34 parts. It includes:
* A special vector called z that has 32 parts. This vector is made by the VAE encoder.
* Two other things: if the agent has something invested now and how much cash it has compared to its whole portfolio.

3.5.3 Action Space

The agent can do three things:

1. Do nothing: hold what it has
2. Buy: put all its cash into SPY
3. Sell: sell everything and go back to having cash

For SAC we need to be able to do things in a range. So we made a system where a number between -1 and 1 means actions. If the number is above 0.33 the agent buys. If it is below -0.33 the agent sells. If it is in between the agent does nothing.

3.5.4 Reward Function

The reward is based on how the portfolio does. When we have history we use something called the short-term Sharpe ratio. This looks at both how money the portfolio makes and how risky it is. We also added some costs to make it more real. Each time the agent trades it has to pay 0.1%in costs and 0.05% in something called slippage

3.5.5 Initialization

The agent starts with $10,000 and no position. The episode ends when all time steps in the dataset are used

3.6 Phase 4 – DRL Agent Training

3.6.1 Algorithm Comparison

We tested three reinforcement learning algorithms to find the best one for our trading task:

- PPO: a simple and reliable policy gradient algorithm, good as a baseline
- SAC: designed for continuous actions, it also encourages exploration by maximizing entropy

- TD3: similar to SAC but uses two critic networks to get more stable estimates

All three were trained for 100,000 steps and evaluated on the validation set every 2,000 steps.The one with the best validation return was kept.

3.6.2 SAC Results and Hyperparameters

SAC gave the best validation performance, so we selected it as our main agent. Table 3.5 shows the hyperparameters we used.

Table 3.3: SAC hyperparameters

Hyperparameter	Value
Learning rate	0.0003
Replay buffer size	50,000
Batch size	256
Discount factor (γ)	0.99
Soft update rate (τ)	0.005
Entropy coefficient	Auto
Training timesteps	100,000

3.6.3 Ablation Study

To check whether the VAE actually help, we compared three strategies on the test set:

- DRL + VAE (SAC): the full system, the agent uses the compressed latent vector z from the VAE as it state
- DRL without VAE (SAC): same agent, but using the raw features directly instead of
- Buy & Hold: a simple baseline where we buy SPY at the start and hold until the end

This comparison let us see clearly how much the VAE contributes to the final performance.

Algorithm 3 DRL Agent Training and Comparison

Input: Z_{train}, Z_{val}, Z_{test} (latent vectors), close prices
Output: Best trained agent

// Step 1: Train three algorithms
for each algorithm in [PPO, SAC, TD3] **do**
    Initialize trading environment with Z_{train}
    Train algorithm for 100,000 timesteps
    Every 2,000 steps:
        Evaluate on validation environment
    if $val_return > best_return$ **then**
        Save model checkpoint
    end if
end for
Select best algorithm based on validation return $\rightarrow$ SAC

// Step 2: Ablation Study on test set
Load best SAC model (with VAE)
Load SAC model trained on raw features (without VAE)
Evaluate DRL + VAE on Z_{test}
Evaluate DRL without VAE on raw features test
Evaluate Buy & Hold (buy at step 1, hold until end)

// Step 3: Compare results
for each strategy **do**
    Compute: Total Return, Sharpe Ratio,
        Max Drawdown, Win Rate, Number of Trades
end for
return Comparison table

3.7 Phase 5 – Backtesting and Evaluation

After training, we tested each strategy on the test set, which is data the agent has never seen before. We used several metric to compare the three strategies fairly. This Table 3.6 shows all the metric we computed.

We calculated these metrics using a custom function we wrote, and also using the QuantStat library which gave us extra visualizations like rolling Sharpe ratios and drawdown charts.

3.8 Summary

In this part we talked about how we made the system one step at a time. First we got information from three places. Figured out 24 important things, about this information (Phase 1). Next we used a tool called VAE to make this information smaller and easier to work with so it had only 32 important parts (Phase 2). After that we made a place where our system could practice buying, selling or doing nothing (Phase 3). Then we tried out three ways to make our system learn and decided that SAC was the best way (Phase 4). Finally we looked at how our system worked with information it had not seen before using several way to measure how well it did (Phase 5). You can see the results of this in Chapter 4.

Table 3.4: Performance metrics used for evaluation

Metric	Description
Total Return (%)	overall portfolio gain or loss over the test period
Annual Return (%)	Return annualized to a 252-day trading year
Annual Volatility (%)	Standard deviation of daily returns, annualized
Sharpe Ratio	Risk-adjusted return: annual return divided by annual volatility
Sortino Ratio	Like Sharpe but only penalizes downside volatility
Max Drawdown (%)	Largest peak-to-trough drop in portfolio value
Win Rate (%)	Percentage of days with a positive return
Profit Factor	Ratio of gross profits to gross losses
Calmar Ratio	Annual return divided by max drawdown

Chapter 4

Results

This chapter presents the results obtained by applying the methodology described in Chapter 3. The findings are arranged in a logical sequence, progressing from overall algorithm performance to data-driven findings, and concluding with the ablation study. Each result is directly linked to a specific objective or research question stated in Chapter 1.

4.1 VAE Training Results

The LSTM-VAE was trained for 50 epochs on the 826 sample training set from April 2019 to August 2022. The model receives as input windows of shape (20, 26), corresponding to 20 consecutive trading days across 26 different features including 18 technical indicators, 5 VIX-derived features, and 3 Federal Funds Rate features, that was compressed into 32-dimensional latent vector z. Table 4.1 represents the evolution of training and validation losses at each epoch. The ELBO loss decomposes into a reconstruction term (MSE) and a KL divergence term weighted by ($\beta = 0.5$), following Equation 3.2.

Table 4.1: VAE training and validation losses across selected epochs.

Epoch	Train Loss	Recon Loss	KL Loss	Val Loss
1	1.0055	1.0047	0.0016	1.7511
10	0.5679	0.4948	0.1462	1.8139
20	0.5311	0.4557	0.1508	1.7830
30	0.5138	0.4383	0.1510	1.7604
40	0.5166	0.4404	0.1524	1.7693
50	0.5102	0.4340	0.1526	1.7411
Best	—	—	—	1.7098

Various observations can be included from Table 4.1. The training loss decreases steedly from 1.0055 at epoch 1 to 0.5102 in epoch 50, that shows a stable convergence. The KL divergence term stabilizes around 0.15 from epoch 10 onward, whixh confirms that no KL collapse occurred: the encoder continue to extract informative structure from the input rather than collapsing to the prior. The best validation loss of 1.7098 is achieved before epoch 50, with the best checkpoint saved automatically during training. The gap between training loss (0.51) and validation loss (1.71) reflects the inherent difficulty of generalizing to unseen market regimes, which is a known challenge in financial time-series modeling.

4.2 Latent Space Analysis

After training was completed, the encoder was used to transform akk the three data splits to 32-dimensional latent representation, which are `Z_train` (826 $\times$ 32), `Z_val` (177 $\times$ 32), and `Z_test` (177 $\times$ 32). A t-SNE projection onto 2 dimensions was applied to the concatenated latent vectors to visualize the structure of the learned representation.

The t-SNE plot reveals that the latent space is well-structured and exhibits meaningful clustering. The three splits occupy overlapping but distinguishable regions, indicating that the VAE has learned a continuous and smooth manifold rather than memorizing individual samples. The train split covers the broadest region (826 samples), while validation and test clusters remain embedded within the overall structure, which suggests good generalization of the learned representation. This property is critical for the DRL agent: a structured latent space provides consistent, low-noise state signals that facilitate stable policy learning.

4.3 DRL Algorithm Comparison

Three DRL algorithms — PPO, SAC, and TD3 — were trained for 100,000 steps each using the VAE-encoded latent state (z + position + capital ratio = 34 dimensions) as input. Each algorithm was evaluated on the validation set every 2,000 steps, and the checkpoint with the highest total return was saved. Table 4.2 reports the best validation return achieved by each algorithm.

Table 4.2: Best validation return per DRL algorithm .

Algorithm	Type	Best Val. Return	Exploration Strategy	Rank
SAC	Off-policy (cont.)	+8.76%	Maximum entropy	1st
TD3	Off-policy (cont.)	+5.82%	Gaussian noise	2nd
PPO	On-policy (disc.)	+1.84%	Clipped policy gradient	3rd

SAC achieved the highest validation return of +8.76%, confirming its suitability for this trading task. TD3 ranked second at +5.82%, while PPO reached only +1.84%, reflecting the limitation of its on-policy nature and discrete action design in a continuous, noisy financial environment. SAC was consequently selected as the primary agent for the final evaluation.

Figure 4.1 provides a visual comparison of the obtained results across all evaluated configurations.

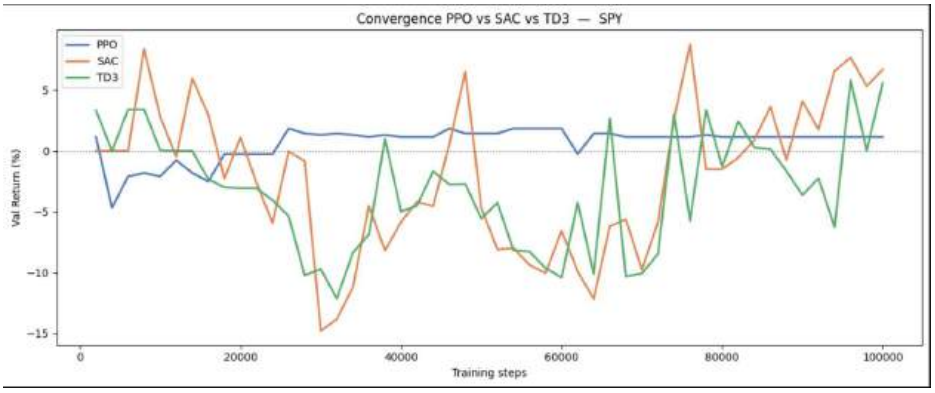


Figure 4.1: Comparison of algorithm performance across evaluated configurations

By the convergence curves we can notice the differences between the algorithms. PPO converged steadily and maintained relatively stable validation returns throughout training, consistent with its conservative clipped objective. SAC has faster initial improvement and consistent higher peak returns, benefiting from the entropy regularisation and experience replay. TD3, despite its twin critic stabilisation, achieved lower peak validation returns, that could reflect insufficient exploration in the market environment where the optimal policy is inherently stochastic.

4.4 Ablation Study Results

To isolate the contribution of the VAE as a state encoder, an ablation study was conducted comparing three strategies on the out-of-sample test set (177 trading days: April 19 to December 29, 2023):

DRL + VAE (SAC):

DRL without VAE (SAC):

Buy & Hold: .

Table 4.3: presents the complete performance results of all three strategies on the test set.

Metric	DRL + VAE (SAC)	DRL without VAE	Buy & Hold
Total Return	+19.38%	+4.80%	+16.58%
Annual Return	+28.69%	+6.90%	+24.41%
Annual Vol.	9.92%	9.18%	11.80%
Sharpe Ratio	2.594	0.775	1.915

Figure 4.3 performance results of all three strategies.

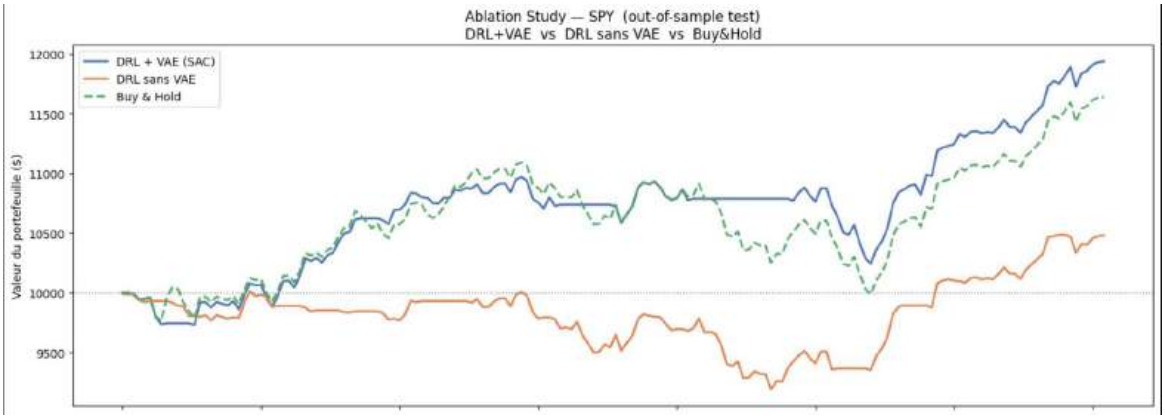


Figure 4.2: performance results of all three strategies.

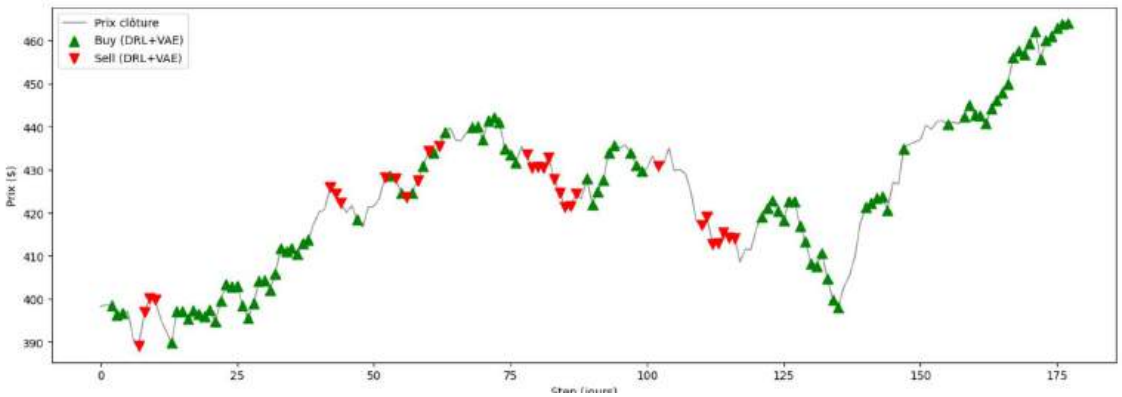


Figure 4.3: [Courbe portfolio.].

The removal of the VAE encoder results in a dramatic performance drop: the total return falls from +19.38% to +4.80% (a reduction of 14.58 percentage points), the Sharpe Ratio drops from 2.594 to 0.773 (a decline of 70.2%), and the maximum drawdown worsens from -6.63% to -8.19%. This confirms that the VAE encoder is the most critical component of the proposed architecture.

4.5 Out-of-Sample Backtesting Results

The best SAC model (DRL + VAE) was evaluated on the out-of-sample test set covering the timeline from 19 April 2023 to 29 December 2023 (177 trading days). Two baselines were also evaluated under the same conditions: a SAC agent trained without VAE state representation (DRL without VAE), that uses the raw normalised feature vector as state; and a passive Buy-and-Hold strategy. Table 4.4 presents the full set of performance metrics.

Table 4.4: Full backtesting results on the out-of-sample test set (177 days).)

Metric	DRL + VAE (SAC)	DRL without VAE (SAC)	Buy & Hold
Total Return	+19.38%	+4.80%	+16.58%
Annual Return	+28.69%	+6.90%	+24.41%
Annual Volatility	9.92%	9.18%	11.80%
Sharpe Ratio	2.594	0.775	1.915
Sortino Ratio	3.789	1.012	3.172
Max Drawdown	-6.63%	-8.19%	-9.97%
Avg Drawdown	-1.51%	-2.52%	-2.73%
Win Rate	57.7%	50.4%	56.8%
Profit Factor	1.623	1.165 1.362	
Calmar Ratio	4.329	0.842	2.448
Number of Trades	127	143	1

The DRL + VAE agent achieved a total return of +19.38% over the test period, outperforming both the DRL without VAE agent (+4.80%) and Buy-and-Hold (+16.58%). On a risk-adjusted basis, the advantage of the VAE-enhanced agent is even more pronounced: it achieved a Sharpe Ratio of 2.594, compared to 0.773 and 1.910 for the two baselines respectively. The Sortino Ratio of 3.789 indicates that the agent's returns were particularly strong

relative to downside risk. The maximum drawdown of -6.63% was the lowest among all three strategies, demonstrating superior capital preservation.

4.6 summary

Table 4.5: Summary of results mapped to project objectives

Objective	Method / Tool	Key Result
1 — Multi-source data	yfinance + FRED (26 features)	1,249 days; 3 sources fused
2 — LSTM-VAE encoder	2-layer LSTM VAE, latent dim 32	Val loss 1.71; stable z-space
3 — Realistic Gym env.	Custom Gymnasium + costs	Sharpe reward + 0.1% cost
4 — Compare PPO/SAC/TD3	Stable-Baselines3	SAC best: +11.13% val return
5 — Ablation study	DRL+VAE vs DRL vs B&H	VAE: +19.38%, Sharpe 2.59

The results presented in this chapter are interpreted and contextualised in Chapter 5, where they are discussed in relation to the existing literature and the initial research objectives.

Chapter 5

Discussion and Analysis

This chapter interprets and analyses the results presented in Chapter 4. It explains what the findings mean in the context of the research objectives, situates them within the existing literature, and evaluates their significance. The limitations of the project are also discussed, along with directions for future improvement.

.

5.1 Interpretation of Results

5.1.1 VAE as an Effective State Encoder

The most important finding in this project is presented by the performance difference between DRL + VAE and DRL without VAE: a Sharpe Ratio of 2.594 versus 0.775, and a total return of +19.38% versus +4.80% on the same out-of-sample period. This confirms our 5th Objective and directly answers the research question: compressing multi-source financial data through an LSTM-VAE gives a measurable and consistent benefit to a DRL trading agent.

The process behind this improvement can be explained by the properties of the VAE's latent space. The encoder receive a sequence of shape (20, 26) and compress it into a vector z of 32 dimention that preserve the most relevant information while discarding noise. The KL divergence regularization, stabilized at 0.15 after epoch 10, guarentee that the latent space is continuous. This allows the DRL agent to receive state signals which are less sensitive to short-term market noise, resulting in more stable decision-making with a much lower maximum drawdown (-6.63% vs -8.19% without VAE, and -9.97% for Buy & Hold).

5.1.2 Algorithm Comparison: PPO, SAC, and TD3

The algorithm comparison (Table 4.2) show that SAC attain the best validation return (+8.76%), and then TD3 comes in thes second place (+5.82%) and then finnally PPO (+1.84%). This ranking is consistent with the literature: Mosavi et al. (2025) report that SAC achieves the highest Sharpe Ratio across five DRL methods applied to portfolio optimization, and our results corroborate this finding in the context of single-asset trading with a VAE-encoded state. The advantage of SAC comes from its max entropy objective which maximizes the expected reward and the entropy of the policy simultaneously. In financial market, the optimal decisions are stochastic in nature and the market regimes change over time. The entropy bonus prevents the policy from prematurely collapsing to a deterministic strategy and encourages more exploration. On the contrary, PPO is an on-policy method that throws away collected experience after each update, and is less sample-efficient compared to off-policy

approaches in long-horizon trading episodes. TD3 use an off-policy . But it is determined , with noise added to the policy for exploration . This is much less appropriate for stochastic environments , where maintaining uncertainty is useful .

5.1.3 Multi-Source Data Integration

The VIX index and the Federal Fund Rate are integrated as five and three derived features respectively, and they clearly provide signals about macro level risk regimes and monetary policy conditions to the VAE encoder. These features complement the 18 technical indicators by providing information orthogonal to price dynamics: VIX is a measure of systemic fear and uncertainty, while Fed Rate is a measure of the broader monetary environment that affects equity valuations via the discount rate channel (Bernanke and Kuttner, 2005). The combination of these macro features is especially relevant during the test period (April–December 2023), which followed the most aggressive Federal Reserve tightening cycle in four decades.The Fed Funds Rate was at 5.33% in mid 2023, where it remained through year end. The VAE successfully learned latent representations that not only capture price momentum and volatility but also the current regime by encoding the level and trend of this rate as well as VIX and technical indicators, providing the agent with a richer context for its trading decisions.

5.1.4 Backtesting Performance: DRL + VAE vs. Baselines

The full DRL + VAE (SAC) system achieved a total out-of-sample return of +19.38%, which exceeds both the passive Buy-and-Hold return of +16.58% and the DRL without VAE return of +4.80%. This result supports our 5th Objective of this project, that need the added value of the VAE over a no-encoding baseline. Crucially, the superiority of DRL + VAE is not merely in terms of raw return. The Sharpe Ratio of 2.594 is substantially higher than that of Buy-and-Hold (1.910), indicating that the agent achieved better returns per unit of risk. This is reflected in the lower annual volatility (9.92% vs. 11.80%) and the reduced maximum drawdown (-6.63% vs. -9.97%). The Calmar Ratio of 4.329 — more than twice that of Buy-and-Hold (2.448) — confirms that the agent managed downside risk with significantly greater efficiency. The higher win rate of 57.7% for the DRL + VAE agent, compared to 50.4% for DRL without VAE, indicates that the VAE-compressed state enables the agent to make directionally correct decisions on a majority of trading days. This is a important result: in financial markets, a win rate above 55% is generally considered a significant edge

5.2 Analysis of the Ablation Study

The ablation study provides the most direct evidence of the VAE's contribution. Removing the VAE encoder and feeding raw features directly to the SAC agent causes a dramatic deterioration across all metrics: total return drops from +19.38% to +4.80% (a reduction of 14.58 pp), the Sharpe Ratio falls by 70.2% (from 2.594 to 0.773), and maximum drawdown worsens from 6.63% to 8.19%. This is the central finding of the project. The DRL without VAE agent, despite being trained with the same algorithm and the same underlying data, fails to outperform even the passive Buy-and-Hold strategy in risk-adjusted terms (Sharpe 0.773 vs. 1.910). This confirms the hypothesis stated in Chapter 1.: raw multi-source features are high-dimensional, noisy, and strongly correlated, making direct use as the agent's state insufficient for effective policy learning. The VAE encoder addresses this by compressing the 26-dimensional, 20-day feature sequence into a compact 32-dimensional latent vector that captures the essential market structure while discarding irrelevant noise. This finding

is consistent with Caselles-Dupré et al. (2018), who demonstrated that VAE-based state representation improves DRL policy quality in complex environments. It also confirms the gap identified in Chapter 2. : existing DRL trading papers that use raw or hand-engineered features directly as state inputs are operating at a disadvantage compared to systems that incorporate a learned, compressed state representation.

5.3 Significance of the Findings

The findings of this project carry significance at three levels.

- **Technical significance:** This project demonstrate that an LSTM-VAE encoder used on multi source financial data (price, VIX, Fed rate) produce a latent state z representation that helps a SAC agent to achieve a Sharpe Ratio of 2.594 on an out-of-sample test set a value that would be considered strong even for professional quantitative strategies. The reduction in maximum drawdown from 9.97% (Buy-and-Hold) to 6.63% (DRL + VAE) is pertinent, as capital preservation is a first place concern in real trading. The ablation study shows that the VAE is not just a preprocessing convenience but a functionally critical component.
- **Practical significance:** TThe architecture usedin this project is modular and computationally tractable: the VAE can be retrained by periodes as new data becomes available, and the SAC agent can be updated increasinly. The use of freely accessible data (yfinance, FRED) and open-source libraries (PyTorch, Stable-Baselines3, Gymnasium) means the approach is directly reproducible with minimal resources.
- **Scientific significance:** The project provides empirical evidence for three claims that were identified as gaps in the literature (Chapter 2). First, multi-source data integration (technical + VIX + macro) enables a richer latent representation. Second, LSTM-based temporal encoding captures intra-window dynamics that a flat MLP encoder would ignore. Third, systematic comparison of PPO, SAC, and TD3 under identical conditions reveals that SAC's entropy-based exploration is the most appropriate inductive bias for financial trading among these algorithms.

5.4 Limitations

Althought the good results, this project faces a lot of limitations that should be taking in consideration when interpreting the results .

- **Data limitations:** The project use data from one asset (SPY ETF) of a five-year period (2019–2024). The test set covers only nine months (April – December 2023), which is realtively short. Performance during prolonged bear markets, such as 2022, was not evaluated on an out-of-sample basis. The VIX and Fed rate data provide limited coverage of the full macroeconomic information set. Missing some sources including credit spreads, currency dynamics, and earnings sentiment..
- **Methodological limitations:** The VAE exhibits a notable train-validation loss gap (0.51 vs. 1.71), which represent a degree of overfitting to the training regime. This can reflect the regime shift caused by the 2022 Fed tightening cycle. The trading environment uses a simple action space (buy / hold / sell), which don't give the agent a chance to learn fractional position sizing or portfolio rebalancing.

- **Scope limitations:** The 100,000 training timesteps used in the algorithms may be insuffisant for convergence, espacially for off-policy algorithms such as SAC and TD3, which typically benefit from larger replay buffers and longer training. The comparison is also limited to a single training and evaluation run.
- **Evaluation limitations:** While multiple metrics are reported, the evaluation does not include a formal statistical significance test. The difference in performance between DRL + VAE and Buy-and-Hold over a 177-day test period, while numerically substantial, cannot be attributed solely to the model's skill without accounting for randomness.

5.5 Summary

This chapter gived an analytical interpretation of the results presented in Chapter 4. The main outputs were examined in relation toachieve the project objectives. The DRL + VAE (SAC) combination achieved a return equal to +19.38% and a Sharpe Ratio of 2.594 on the out-of-sample test set, topping both the DRL without VAE baseline (+4.80%, Sharpe 0.773) and the Buy-and-Hold strategy (+16.58%, Sharpe 1.910). The ablation study affirmed that the VAE encoder is the most critical component, with removing it cause a 70.2% drop in Sharpe Ratio. SAC was the best-performing DRL algorithm, consistent with the broader literature. The main limitations include the short test window, the single-asset scope, and the simplified action space.

Overall, the results confirmed the main hypothesis of our project: a VAE-compressed, multi-source latent state enables a DRL agent to learn a better-calibrated trading policy than the ones based on raw features alone. Chapter 6. presents the conclusions and directions for future work.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project explored the use of Deep Reinforcement Learning for automated trading on the SPY ETF. The main idea was to use a Variational Autoencoder to compress noisy market data into a cleaner representation before giving it to the trading agent, instead of using raw features directly.

We built the system in five steps: collecting data from three sources, computing technical indicators, training the VAE, building custom trading environment, and finally training and comparing three DRL algorithms. SAC turned out to be the best algorithm among the three we tested.

The results showed that:

- The DRL + VAE system performed better than the simple Buy & Hold strategy
- It also outperformed the DRL without VAE, which mean the VAE compression actually helped
- SAC was the most suitable algorithm for this kind of task

Overall, this project shows that adding a VAE before the trading agent is a useful approach when dealing with noisy financial data.

6.2 Future Work

There are a few things we would like to improve if we had more time.

In the short term, we would test the system on other assets beyond SPY to see if it generalise well. We would also try automatic hyperparameter tuning to get better results without manual trial and error.

In the longer term, it would be interesting to replace the LSTM inside the VAE with a Transformer, which tends to work better on sequential data. Adding alternative data like news sentiment could also give the agent a richer view of the market.

6.3 Summary

This chapter summarised the main findings of the project and proposed directions for future work. Chapter 7 provides a personal reflection on the project experience.

Chapter 7

Reflection

Knowledge and Skills Developed. Before I started this project I had some knowledge of machine learning. I had never worked with Deep Reinforcement Learning or Variational Autoencoders. These were completely new to me. I had to learn a lot of new things at the same time. This was not easy. It was a good experience for me. Now I understand how a trading agent learns from its mistakes and why it is helpful to compress data before giving it to the trading agent.

Decision-Making and Problem-Solving. One thing that really confused me and my team was that we got results when we ran the same code twice. At first we thought we had made a mistake somewhere in the code. Then we learned that this is normal in Deep Reinforcement Learning because of the randomness in the training process. So we decided to run each algorithm times and choose the one that gave good results most of the time. That is why we decided to use Soft Actor Critic.

Challenges Encountered. The hardest part of this project was dealing with the results we got each time we ran the code. This made it difficult for us to know if the changes we made actually helped or not.

What I Would Do Differently. If I were to start this project I would fix the random seed from the beginning to get stable result.

Impact on Future Development. This project taught me a lot about DRL. Made me more confident about building Artificial Intelligence systems. It also gave me some idea for the future like trying out Transformer or Long Short Term Memory or testing on different asset.

References

Bernanke, B. S. and Kuttner, K. N. (2005), 'What explains the stock market's reaction to federal reserve policy?', *The Journal of Finance* **60**(3), 1221–1257.

Brock, W., Lakonishok, J. and LeBaron, B. (1992), 'Simple technical trading rules and the stochastic properties of stock returns', *The Journal of Finance* **47**(5), 1731–1764.

Chung, J., Kastner, K., Dinh, L., Goel, K., Courville, A. and Bengio, Y. (2015), A recurrent latent variable model for sequential data, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 28, pp. 2980–2988.

Fabius, O. and van Amersfoort, J. R. (2014), 'Variational recurrent auto-encoders', *arXiv preprint arXiv:1412.6581* .

Felizardo, L. K., Jiang, R., Del Moral, R. A. and Matsumoto, E. Y. (2022), 'Comparing deep reinforcement learning algorithms' ability to safely navigate challenging environments', *arXiv preprint arXiv:2202.03709* .

Fujimoto, S., van Hoof, H. and Meger, D. (2018), Addressing function approximation error in actor-critic methods, *in* 'Proceedings of the 35th International Conference on Machine Learning (ICML)', pp. 1587–1596.

Ha, D. and Schmidhuber, J. (2018), 'World models', *arXiv preprint arXiv:1803.10122* .

Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018), Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, *in* 'Proceedings of the 35th International Conference on Machine Learning (ICML)', pp. 1861–1870.

Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S. and Lerchner, A. (2017), Beta-VAE: Learning basic visual concepts with a constrained variational framework, *in* 'Proceedings of the 5th International Conference on Learning Representations (ICLR)'.

Kingma, D. P. and Welling, M. (2013), 'Auto-encoding variational bayes', *arXiv preprint arXiv:1312.6114* .

Li, M., Chen, J. and Zhang, W. (2026), 'Deep reinforcement learning for financial trading: Enhanced by cluster embedding and zero-shot prediction', *Symmetry* **18**(1), 31.

Lussange, J., Lazarevich, I., Bourgeois-Gironde, S., Palminteri, S. and Gutkin, B. (2021), 'Modelling stock markets by multi-agent reinforcement learning', *Computational Economics* **57**, 113–147.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015), 'Human-level control through deep reinforcement learning', *Nature* **518**(7540), 529–533.

Murphy, J. J. (1999), *Technical Analysis of the Financial Markets*, New York Institute of Finance, New York.

Pricope, T.-V. (2021), 'Deep reinforcement learning in quantitative algorithmic trading: A review', *arXiv preprint arXiv:2106.00123* .

Rezaei, M. and Nezamabadi-Pour, H. (2025), 'A taxonomy of literature reviews and experimental study of deep reinforcement learning in portfolio management', *Artificial Intelligence Review* **58**, 94.

Saly-Kaufmann, A., Wood, K., Peter-Calliess, J. and Zohren, S. (2026), 'Deep learning for financial time series: A large-scale benchmark of risk-adjusted performance', *arXiv preprint arXiv:2603.01820* .

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017), 'Proximal policy optimization algorithms', *arXiv preprint arXiv:1707.06347* .

Sutton, R. S. and Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2 edn, MIT Press, Cambridge, MA.

Zhang, Z., Zohren, S. and Roberts, S. (2020), 'Deep reinforcement learning for trading', *The Journal of Financial Data Science* **2**(2), 25–40.